

Webhook Setup — Runbook

Payment Failure Recovery Engine · keep next to the repo

The one-paragraph version

A webhook inverts the usual direction: instead of you calling Razorpay, **Razorpay calls you**. You register a URL once; from then on Razorpay POSTs to it whenever a payment event happens. This engine is entirely reactive — it does nothing until a `payment.failed` event arrives. The webhook is the trigger for all five layers. No webhook, no input, nothing runs.

Two ways to run it

Path A — fully local (no Razorpay account, no internet)

```
uvicorn src.main:app --reload          # terminal 1
python scripts/simulate_webhooks.py --count 20  # terminal 2
```

The script builds real Razorpay-shaped payloads and HMAC-signs them, so they pass verification exactly like genuine ones. Use this for demos — it is fast and never flakes.

The gotcha that costs 20 minutes. `simulate_webhooks.py` defaults to `--secret test_webhook_secret_123`. If `.env`'s `RAZORPAY_WEBHOOK_SECRET` differs, every request returns **401** with no useful error. Either set that value in `.env` for local work, or pass `--secret <your-secret>`.

Path B — real Razorpay test mode

```
uvicorn src.main:app --reload
ngrok http 8000          # copy the https://... URL it prints
```

`localhost:8000` exists only inside your laptop — Razorpay's servers cannot reach it. ngrok gives you a public HTTPS URL and forwards traffic down to your local port.

Dashboard → **Account & Settings** → **Webhooks** → **Add New Webhook**:

Field	Value
Webhook URL	<code>https://<ngrok-url>/webhooks/razorpay</code>
Secret	Invent any random string — put the identical string in <code>.env</code>
Active events	<code>payment.failed</code> and <code>payment.captured</code> only

The free ngrok URL **changes on every restart** — re-paste it each time. Its inspector at `http://localhost:4040` shows every incoming request with full headers and body.

Where the three keys come from

.env variable	Source
<code>RAZORPAY_KEY_ID</code>	Dashboard → API Keys (Test Mode on) → Generate. Starts <code>rzp_test_</code>
<code>RAZORPAY_KEY_SECRET</code>	Shown once at generation — copy immediately
<code>RAZORPAY_WEBHOOK_SECRET</code>	Not the key secret. You invent it in the Add Webhook form

`src/config.py` reads all three automatically via pydantic-settings. No code edit anywhere. `.env` is gitignored, so keys stay out of the repo.

What happens to one webhook

POST `/webhooks/razorpay` → `receive_razorpay_webhook` in `src/ingestion/router.py`

#	Line	Action
1	:103	Read raw bytes of body — before any parsing
2	:106	No signature header → 401 , stop
3	:110	HMAC mismatch → 401 , stop. Nothing untrusted passes this line
4	:113	Now safe to parse JSON
5	:125	Build event ID: <code>{type}_{payment_id}_{created_at}</code>
6	:135	Already seen? → 200 Already processed , stop
7	:147	Store raw event in Postgres — append-only, replayable
8	:152	Queue background processing
9	:162	Return 200 OK immediately

Why 8 and 9 are split from the real work: Razorpay expects a response in seconds, but the pipeline calls an LLM, the DB, and the Payment Links API. Respond too slowly and Razorpay treats delivery as failed and **re-sends** — you would process the same failure twice. So the endpoint does only verify → dedupe → store → acknowledge, and hands off. Step 6 exists precisely because those re-sends are real.

`payment.captured` means the payment eventually succeeded, so any still-pending retry is marked **superseded** — that is what stops you charging someone who already paid.

Why the signature exists

Your endpoint is a public URL — anyone can POST to it. Without verification, a stranger could send a fake `payment.failed` and make the engine issue retries for payments that never existed. The secret is never sent over the wire; Razorpay sends `HMAC-SHA256(secret, raw_body)` in `X-Razorpay-Signature` and you recompute it.

Two details worth defending in the panel — both already correct in `src/ingestion/signature.py`:

- **Raw bytes, never re-serialized JSON.** Parsing then re-dumping can shift whitespace or key order by one byte and the HMAC breaks. The code reads `await request.body()` first.
- **Constant-time compare.** `hmac.compare_digest`, not `==`. A normal comparison exits at the first wrong character, leaking through timing how many leading bytes were guessed right — enough to recover a valid signature byte by byte.

Troubleshooting

Symptom	Cause & fix
401 Invalid signature	Secret in <code>.env</code> ≠ secret in dashboard (or in <code>--secret</code>). They must match exactly
401 Missing signature	Request did not come from Razorpay, or a proxy stripped the header
Nothing arrives at all	ngrok restarted and the URL changed — re-paste it into the dashboard

200 Already processed	Working as intended — dedup caught a re-delivery
200 OK but no pipeline activity	Background task threw. Check uvicorn logs; DB likely unreachable
Event stored, never processed	Event type is neither <code>payment.failed</code> nor <code>payment.captured</code>

Files that matter

Path	Role
<code>src/ingestion/router.py</code>	The endpoint — verify, dedupe, store, hand off
<code>src/ingestion/signature.py</code>	HMAC verification
<code>src/ingestion/idempotency.py</code>	Duplicate-event check
<code>src/orchestrator.py</code>	Runs the five layers after the webhook lands
<code>scripts/simulate_webhooks.py</code>	Local signed-webhook generator
<code>.env</code>	The three keys. Gitignored

Before submitting. Get at least one *real* test-mode webhook through end to end and keep the log line. The brief calls out "real infrastructure, not a mock" — a mocked gateway reads as a toy. Demo with the simulator for reliability, but be able to prove Path B works.

The eval harness needs none of this — `python -m eval.runner --skip-llm` runs standalone with no keys, no database, no internet. Only the live webhook path needs credentials.